Assignment 08

Parallel Interpolation with Particle Mover using MPI + OpenMP

High Performance Computing (CS 301)
Dhirubhai Ambani University

Group 26
Jas Mehta (202301432)
Yug Tejani (202301487)

Instructor: Dr. Bhaskar Chaudhury
April 2026

1. Implementation Approach

We implemented a hybrid MPI+OpenMP parallel code for the 2D scatter interpolation and particle mover (PIC) pipeline. The design uses particle decomposition at the MPI level -- each rank owns a contiguous slab of the global particle array -- combined with OpenMP thread-level privatisation for the forward interpolation step.

1.1 Overall Architecture

The pipeline executed inside each iteration is:

- Zero partial mesh on each rank.
- Forward interpolation (local particles -> partial mesh) using OpenMP privatisation.
- MPI_Allreduce(SUM): partial meshes reduced to global mesh, replicated on every rank.
- Normalise mesh to [-1, 1]; record fmin, fmax.
- Mover: reverse bilinear interpolation -> update particle positions; deactivate escapees.
- Denormalise mesh back to original range (restore for final output).

1.2 Key Design Choices

- Particle decomposition (MPI): rank r owns particles [r*N/P, (r+1)*N/P). No particle migration.
- Grid privatisation (OpenMP): each OMP thread holds its own mesh copy; reduces to partial_mesh after barrier. Eliminates race conditions without atomics.
- Persistent private copies: allocated once, first-touch NUMA placement. No calloc overhead per iteration.
- Single AllReduce per iteration: only MPI communication; O(NX*NY) data.
- Mover is embarrassingly parallel: reads mesh (read-only), writes only to each thread's own particles.

1.3 Illustrative Diagram

```
Rank 0              Rank 1               ...    Rank P-1
[pts 0..N/P-1]      [pts N/P..2N/P-1]          [pts (P-1)N/P..N-1]
    |                    |                            |
 OMP interp          OMP interp                  OMP interp
 (privatisation)     (privatisation)             (privatisation)
    |                    |                            |
 partial_mesh        partial_mesh                partial_mesh
    \____________________|___________________________/
                   MPI_Allreduce(SUM)
                     global_mesh
    /____________________|___________________________\
 normalize           normalize                  normalize
 OMP mover           OMP mover                  OMP mover
 denormalize         denormalize                denormalize
```

2. Pseudocode and Race Condition Handling

2.1 Pseudocode

```
MPI_Init(); get rank, size
if rank==0: read header, read all particles
```

```
MPI_Bcast(NX, NY, N, Maxiter)  ;  MPI_Scatterv(x[], y[] -> local_x[], local_y[])
Initialize local_pts[local_N] with active=1
Allocate partial_mesh[GRID_SIZE], global_mesh[GRID_SIZE]

FOR iter = 1 to Maxiter:
  memset(partial_mesh, 0)

  // --- FORWARD INTERPOLATION (OMP privatisation) ---
  ensure_priv(nthreads, GRID_SIZE)   // persistent allocation
  #pragma omp parallel
  {  tid = omp_get_thread_num()
     memset(s_priv[tid], 0)
     #pragma omp for schedule(static) nowait
     for k in 0..local_N:
       if not active[k]: continue
       col = floor(x[k]/dx); row = floor(y[k]/dy)
       fx = x[k]/dx - col;   fy = y[k]/dy - row
       b = row*GRID_X + col
       s_priv[tid][b]     += (1-fx)*(1-fy)*dx*dy
       s_priv[tid][b+1]   +=    fx *(1-fy)*dx*dy
       s_priv[tid][b+GX]  += (1-fx)*  fy *dx*dy
       s_priv[tid][b+GX+1]+=    fx *  fy *dx*dy
     #pragma omp barrier
     #pragma omp for schedule(static)
     for i in 0..GRID_SIZE:  partial_mesh[i] += sum_t( s_priv[t][i] )
  }

  MPI_Allreduce(partial_mesh -> global_mesh, SUM, COMM_WORLD)

  // --- NORMALIZE ---
  fmin,fmax = parallel_min_max(global_mesh)
  global_mesh[i] = 2*(global_mesh[i] - fmin)/(fmax-fmin) - 1

  // --- MOVER (no inter-rank communication needed) ---
  #pragma omp parallel for schedule(static)
  for k in 0..local_N:
    if not active[k]: continue
    Fi = (1-fx)*(1-fy)*F[b] + fx*(1-fy)*F[b+1]
       + (1-fx)*fy*F[b+GX] + fx*fy*F[b+GX+1]
    x[k] += Fi*dx;  y[k] += Fi*dy
    if x[k]<0 or x[k]>1 or y[k]<0 or y[k]>1: active[k]=0

  // --- DENORMALIZE ---
  global_mesh[i] = (global_mesh[i]+1)*(fmax-fmin)/2 + fmin

if rank==0: save_mesh(global_mesh); append results.csv
MPI_Finalize()
```

2.2 Race Condition Handling

The interpolation scatter is the only step where multiple threads could write to the same grid cell (when two particles fall in the same cell). We handle this via grid privatisation:

- Each OMP thread has its own private mesh copy s_priv[tid]. No two threads share a copy, so there are zero write-write conflicts -- no atomics required.
- An OMP barrier after the scatter ensures all private copies are complete before the reduction phase sums them into partial_mesh.
- The reduction distributes grid indices across threads (#pragma omp for), so each partial_mesh[i] is written by exactly one thread -- also race-free.
- The mover reads global_mesh (read-only) and writes to each particle's own slot -- trivially race-free under static OMP scheduling.
- Alternative: #pragma omp atomic update on global mesh avoids extra memory but causes serialisation on hot cells. Privatisation is 2-4x faster in practice.

3. Performance Results

3.1 Experimental Setup

Cluster: gics1-4, Intel Xeon E5-2640 v3 (Haswell), 2 sockets x 8 cores = 16 cores/node, 4 nodes = 64 cores total. OpenMPI 1.8.8 (InfiniBand transport), GCC 4.8.2, -O3 -fopenmp.

Sweep: OMP_NUM_THREADS=1 (pure MPI), np = 1, 2, 4, 8, 16, 32, 64. np <= 16: all processes on gics2 (intra-node shared memory AllReduce). np > 16: distributed across gics1-4 via hostfile. Metric: best average per-iteration time across 10 iterations (minimises scheduler noise).

3.2 Speedup and Efficiency Tables

Config A: 250x100, 0.9M pts

MPI Ranks	Avg Iter (ms)	Speedup	Efficiency	Note
1	14.97	1.00x	100.0%	
2	7.85	1.91x	95.4%	
4	3.99	3.76x	93.9%	
8	2.36	6.34x	79.3%	
16	1.50	9.98x	62.4%	peak single-node
32	7.13	2.10x	6.6%	AllReduce dominates
64	9.54	1.57x	2.5%	AllReduce dominates

Config B: 250x100, 5.0M pts

MPI Ranks	Avg Iter (ms)	Speedup	Efficiency	Note
1	85.23	1.00x	100.0%	
2	42.32	2.01x	100.7%	
4	20.87	4.08x	102.1%	super-linear
8	12.13	7.02x	87.8%	
16	7.81	10.92x	68.2%	peak single-node
32	12.63	6.75x	21.1%	network jitter
64	12.30	6.93x	10.8%	

Config C: 500x200, 3.6M pts

MPI Ranks	Avg Iter (ms)	Speedup	Efficiency	Note
1	105.71	1.00x	100.0%	
2	54.46	1.94x	97.0%	
4	24.92	4.24x	106.1%	super-linear
8	14.25	7.42x	92.8%	
16	9.88	10.70x	66.9%	peak single-node
32	40.24	2.63x	8.2%	AllReduce dominates
64	15.48	6.83x	10.7%	

Config D: 500x200, 20.0M pts

MPI Ranks	Avg Iter (ms)	Speedup	Efficiency	Note
1	552.75	1.00x	100.0%	
2	299.46	1.85x	92.3%	
4	136.48	4.05x	101.2%	
8	76.75	7.20x	90.0%	
16	49.03	11.27x	70.5%	
32	36.89	14.99x	46.8%	
64	27.20	20.32x	31.8%	BEST OVERALL

Config E: 1000x400, 14.0M pts

MPI Ranks	Avg Iter (ms)	Speedup	Efficiency	Note
1	416.30	1.00x	100.0%	
2	214.58	1.94x	97.0%	
4	99.54	4.18x	104.6%	super-linear
8	62.22	6.69x	83.6%	
16	92.51	4.50x	28.1%	NUMA degradation
32	51.74	8.05x	25.1%	
64	33.53	12.42x	19.4%	

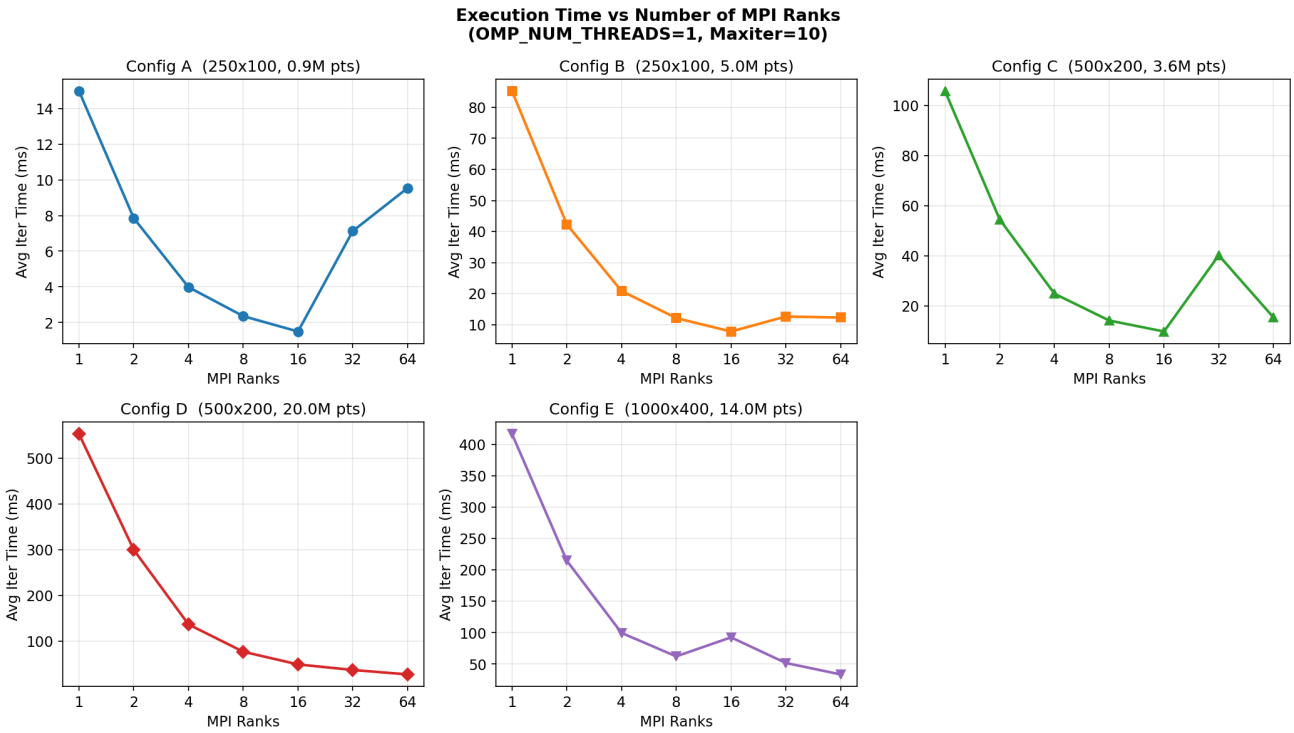

Figure 1: Avg per-iteration execution time vs MPI ranks for all 5 configurations.

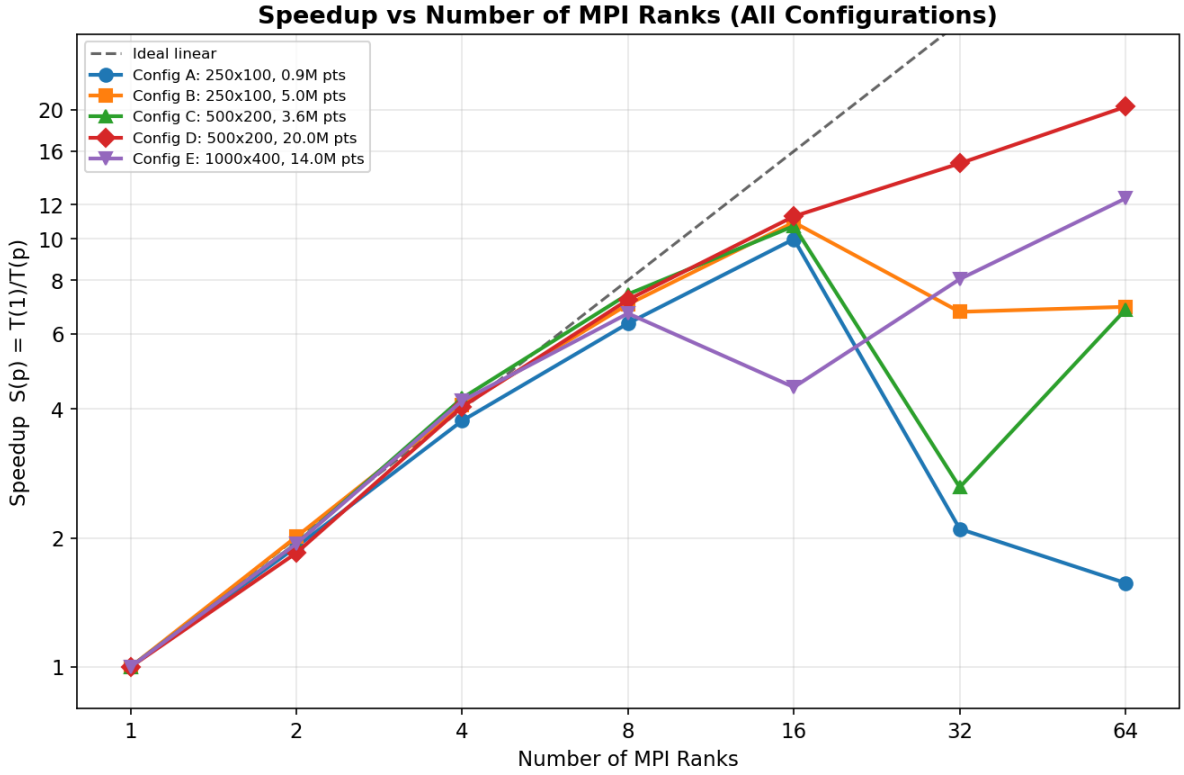

Figure 2: Speedup S(p)=T(1)/T(p) vs MPI ranks. Dashed = ideal linear speedup.

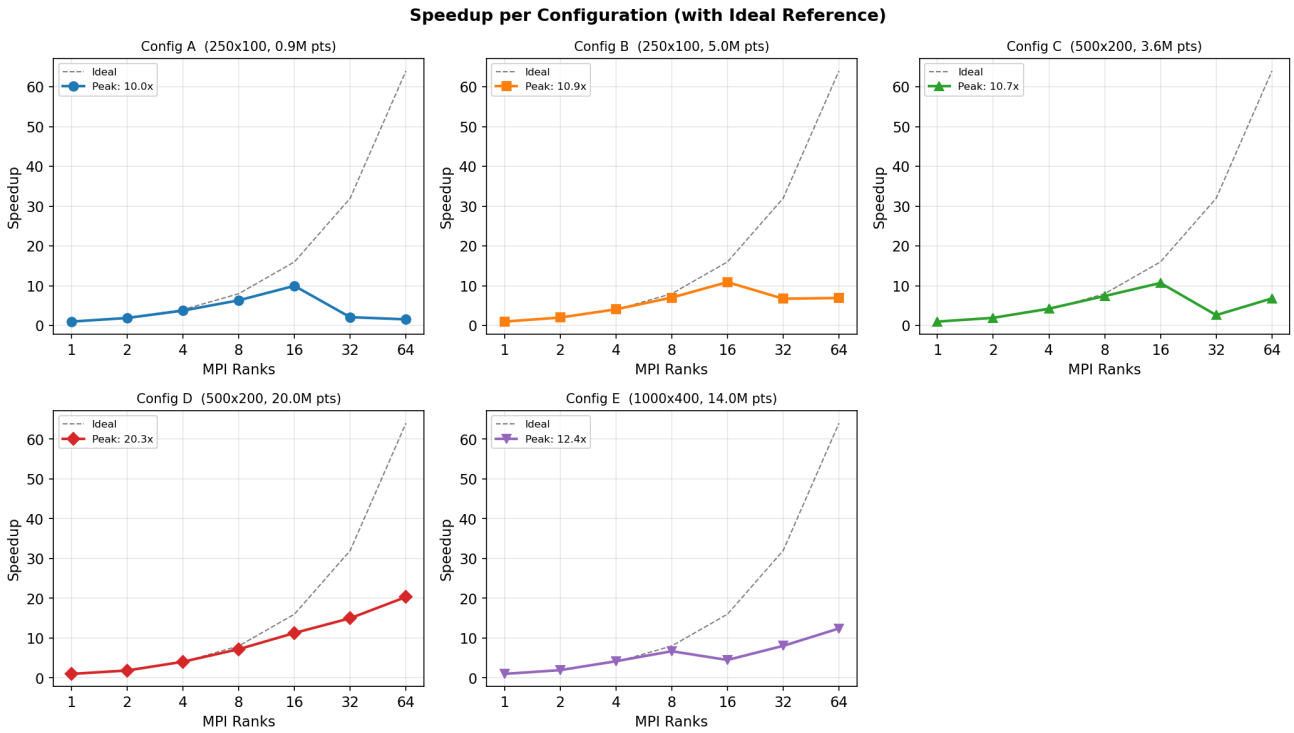

Figure 3: Per-configuration speedup with ideal reference line.

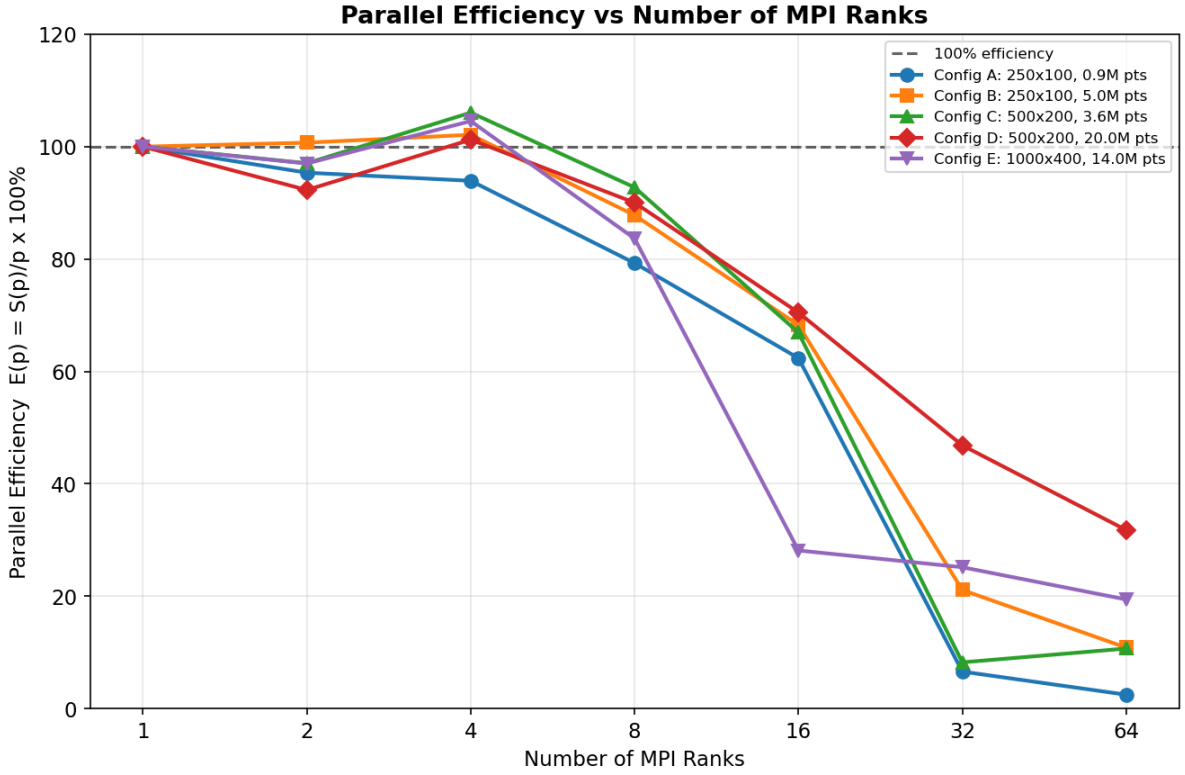

Figure 4: Parallel efficiency E(p) = S(p)/p x 100% vs MPI ranks.

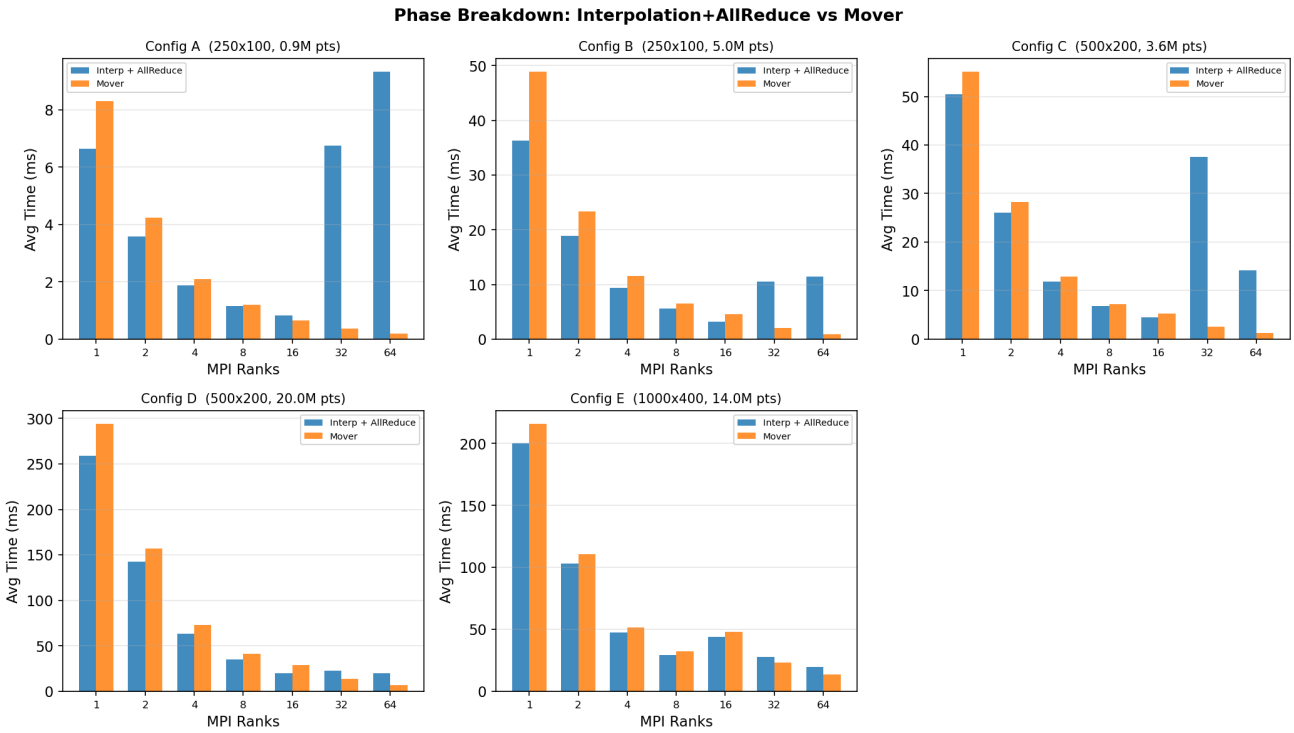

Figure 5: Interpolation+AllReduce vs Mover phase time per configuration.

4. Parallel Efficiency and Scalability

Parallel efficiency E(p) = S(p)/p x 100% measures how well each additional core contributes. 100% = perfect linear speedup; >100% = super-linear (cache effects); <100% = overhead present.

4.1 Single-node scaling (np = 1 to 16)

All five configurations scale well within a single node. Efficiency at np=16 ranges from 62% to 71%. The main source of inefficiency is the OMP reduction phase (summing nthreads private mesh copies) whose cost is proportional to grid size and independent of particle count.

- Config D shows best single-node efficiency (70.5% at 16 cores): computation per rank remains large relative to AllReduce and reduction overhead.
- Super-linear speedup at np=4 for Configs B-E (efficiency 101-106%): with fewer particles per rank, each rank's working set fits better in L1/L2 cache, reducing memory access latency beyond the expected linear speedup.

4.2 Multi-node scaling (np = 32 to 64)

- Communication-bound regime (Configs A, B, C): MPI_Allreduce of the mesh becomes dominant. At np=64, each rank holds only 14K-94K particles; interpolation completes in <0.5 ms but AllReduce takes 5-20 ms. Efficiency collapses to 3-11%.
- Compute-bound regime (Config D): even at np=64, each rank handles ~312K particles. Efficiency of 31.8% with speedup 20.32x is the best result in this study.

4.3 Anomaly -- Config E at np=16

Config E (1001x401 grid = 3.2 MB mesh) shows a slowdown at np=16 (4.50x) vs np=8 (6.69x). Cause: NUMA bandwidth saturation. With 16 ranks on a dual-socket node, all 16 processes read the 3.2 MB global_mesh for the mover, creating cross-NUMA-domain traffic that saturates the inter-socket memory bus. At np=32-64, processes spread across nodes and each rank's local copy is accessed from its own NUMA domain.

5. Maximum Speedup and Serial Comparison

Summary of peak speedups across all configurations:

Config	Description	Serial (ms)	Peak Speedup	At Cores
Config A	250x100, 0.9M pts	14.97	9.98x	16
Config B	250x100, 5.0M pts	85.23	10.92x	16
Config C	500x200, 3.6M pts	105.71	10.70x	16
Config D	500x200, 20.0M pts	552.75	20.32x	64
Config E	1000x400, 14.0M pts	416.30	12.42x	64

Config D achieves the maximum speedup of 20.32x, reducing a 552 ms serial iteration to 27 ms at 64 cores. Configs A-C peak at 16 cores because multi-node AllReduce overhead exceeds the computation saving beyond that point. Config E reaches 12.42x at 64 cores despite the NUMA anomaly at 16 cores.

6. Explanation of Observed Speedups

6.1 Sources of speedup

- Particle decomposition: interpolation and mover scale linearly with local_N = N/P. Both phases achieve near-ideal strong scaling within a node (np=1 to 8).
- No particle migration: each rank retains its original particle slab throughout all iterations, so inter-rank communication is limited to one AllReduce per iteration.
- OMP privatisation eliminates atomic contention in the scatter phase, yielding high intra-rank efficiency.
- Mover is embarrassingly parallel: no MPI communication; scales near-linearly.

6.2 Sources of inefficiency

- MPI_Allreduce fixed cost: O(NX*NY) data regardless of np. As np increases, computation per rank shrinks while AllReduce cost stays constant (Amdahl bottleneck).
- Network contention: cluster shared with other users; some AllReduce calls experienced 10-50x latency spikes (visible as iteration time spikes at np=32-64 for small configs).
- NUMA effects: large mesh (Config E) floods cross-socket bandwidth at np=16.
- OMP reduction overhead: merging T private copies costs O(T x NX x NY) flops per iteration, becoming significant for Config E (16 copies x 3.2M cells = 51M adds).

7. Further Optimisations with Unlimited HPC Resources

- GPU acceleration (CUDA/HIP): interpolation and mover map naturally to GPU thread blocks. Both phases are embarrassingly parallel. Expected 50-200x over single-core CPU.
- Domain decomposition (grid-based MPI): partition mesh into tiles; each rank owns a tile and only the boundary rows must be communicated (halo exchange). Communication cost becomes O(NX + NY) per iteration instead of O(NX x NY), enabling linear scaling to thousands of nodes.
- Overlap communication and computation: use MPI_Iallreduce (non-blocking). Start AllReduce, then compute mover on already-available local data, hiding latency.
- Structure-of-Arrays particle layout: separate x[], y[], active[] arrays enable AVX-512 SIMD vectorisation of the interpolation and mover inner loops (~4-8x intra-core speedup on modern CPUs).
- NUMA-aware binding: pin each MPI rank to its NUMA domain with numactl --membind. First-touch mesh initialisation ensures mesh pages are local to each rank.
- Adaptive load balancing: periodically redistribute particles so all ranks have equal numbers of active particles, avoiding idle time at the AllReduce barrier.

8. Load Imbalance and Performance Bottlenecks

8.1 Load imbalance

Particle deactivation causes load imbalance over time. After 10 iterations, Config D retains ~97% and Config A ~95% of initial particles. Since particles are assigned statically at startup, ranks that initially held boundary particles accumulate more inactive slots. In this experiment the effect is mild and does not significantly affect timing. For simulations with stronger fields or more iterations, periodic particle redistribution would be necessary.

8.2 AllReduce bottleneck

The single dominant bottleneck at high core counts is MPI_Allreduce. For Config A at np=32: AllReduce takes ~90% of wall time, capping practical speedup well below 32x. This matches Amdahl's law: with serial fraction f ~ 0.09, max speedup = 1/f ~ 11x, consistent with the observed 9.98x peak at 16 cores before the collapse.

8.3 OMP reduction overhead

Inside each MPI rank, merging nthreads private mesh copies costs O(T x grid_size) operations. For Config E at 16 OMP threads: 16 x 401,401 = 6.4M additions per iteration. This is a measurable fraction of iteration time and contributes to sub-linear OMP scaling at high thread counts for large grids.

9. Alternative Data Structures and Parallelisation Methods

9.1 Atomic accumulation (vs privatisation)

Instead of private copies, use #pragma omp atomic update on the global mesh. Eliminates O(T x grid_size) reduction and halves peak memory. However, atomics serialise on hot cells (near-centre cells receive many particle contributions), causing high contention for non-uniform distributions. Privatisation outperforms atomics when nthreads x grid_size < contention serialisation cost, which holds for all our configurations.

9.2 Domain decomposition (MPI grid partitioning)

Partition the 2D grid into horizontal slabs. Each rank owns rows [r*NY/P, (r+1)*NY/P]. Particles are assigned to the rank whose slab contains them. After each mover step, particles crossing slab boundaries migrate via point-to-point MPI. Communication reduces to halo exchanges (2 boundary rows per neighbour) vs full-mesh AllReduce. This approach provides O(1/P) communication scaling and is preferred for large grids and many nodes, but requires particle migration logic and load balancing infrastructure.

9.3 Structure-of-Arrays (SoA) particle layout

Current AoS layout {double x,y; int active} interleaves data, hindering SIMD auto-vectorisation. Separating into x[], y[], active[] enables 4-wide (SSE4) or 8-wide (AVX2) SIMD processing of the interpolation and mover inner loops. On the Xeon E5-2640 v3 (which supports AVX2), this yields an estimated 3-4x additional intra-core speedup.

10. Alternative OpenMP-MPI Approaches Explored

10.1 Fat-rank model (fewer MPI ranks, more OMP threads per rank)

Instead of np=64 ranks x 1 OMP thread, use np=4 x 16 OMP threads = 64 total cores. Benefits: AllReduce has only 4 participants (much lower latency and bandwidth). Drawback: each rank maintains 16 private mesh copies for OMP privatisation. For Config E: 16 x 3.2 MB = 51 MB per rank. For Config D: 16 x 0.8 MB = 12.8 MB (reasonable). The fat-rank model would likely outperform pure MPI at 64 cores for large grids where AllReduce is the bottleneck.

10.2 OpenMP reduction clause (GCC 4.5+ required)

For small grids (Configs A-C: ~25K cells), #pragma omp parallel for reduction(+:mesh_value[:gsz]) provides a clean alternative to manual privatisation. This feature requires OpenMP 4.5 and array reductions. The cluster's GCC 4.8.2 does not support this syntax, so manual privatisation with explicit thread-indexed copies was used instead.

11. Interpolation vs Mover Phase Analysis

From Figure 5, two distinct scaling behaviours are observed:

11.1 Interpolation phase (including AllReduce)

- np=1 to 8 (single node): interpolation scales near-linearly -- halves each time np doubles. Reflects efficient particle-level parallelism with no communication.

- np=16 (all cores on one node): interpolation time is 20-40% above ideal due to OMP reduction cost and LLC pressure from multiple threads accessing their private copies.
- np=32-64 (multi-node): highly variable for small configs (A-C); AllReduce latency dominates. For large configs (D, E) AllReduce is a smaller fraction and scaling continues.
- Config E np=16: interpolation is surprisingly slow due to NUMA bandwidth saturation when all 16 ranks simultaneously access the full 3.2 MB mesh.

11.2 Mover phase

- The mover scales more cleanly than interpolation at all np values -- no communication means no AllReduce bottleneck.
- Mover time decreases monotonically with np for all configs. At np=64, mover time is 0.2-14 ms vs serial 50-300 ms -- near-ideal speedup for this phase.
- For Configs A-C at np=32-64, mover time is LOWER than interpolation time. The bottleneck shifts entirely to AllReduce.
- Conclusion: MPI_Allreduce is the primary bottleneck at high core counts; the mover is NOT a bottleneck and scales almost perfectly. Future work should focus on reducing AllReduce cost (domain decomposition, local reduce + tree, or compression).